Go Backend Developer — Practical Training Assignment

Goal: Take a developer from basic Go knowledge to being able to understand, develop, test, debug, and safely contribute to a production Go backend.

Training philosophy:

> Learn → Build → Break → Debug → Refactor → Test → Review → Improve

The developer should maintain **one continuous project** throughout the assignments rather than creating unrelated programs.

Project: Training Management API

Build a backend system for managing:

- Companies
- Departments
- Users
- Training courses
- User training enrollment

The project should gradually evolve from a simple Go program into a production-style REST API.

Assignment 0 — Go & Development Environment Setup

Objective

Understand the Go development environment and write your first Go programs.

Tasks

1. Install:
    - Go
    - Git
    - VS Code
    - Go extension for VS Code
    - Postman/Bruno
2. Configure `GOPATH`, `GOROOT` and Go environment.
3. Create workspace:

```
go-training/
```

4. Initialize Go module:

```
go mod init github.com/<your-name>/go-training
```

5. Understand:

```
go.mod
go.sum
package
import
func
main()
```

6. Write programs for:

Program 1 — Hello Go

```
Hello, Go!
```

Program 2 — Variables

Practice:

```
var
const
:=
```

Program 3 — Functions

Create:

```
add()
subtract()
```

multiply()
divide()

Program 4 — Arrays and Slices

Create a list of employees and perform:

- Add employee
- Remove employee
- Search employee
- Print employees

Program 5 — Maps

Create:

map[string]int

to store:

Employee Name → Age

Program 6 — Struct

Create:

```
type Employee struct {
    ID          int
    Name        string
    Email       string
    Department string
}
```

Program 7 — Pointer

Understand:

&
*

by modifying an employee using a pointer.

Program 8 — Error Handling

Create a function:

GetEmployee(id int) (*Employee, error)

Handle:

employee found
employee not found

Program 9 — JSON

Convert:

Employee

to JSON and JSON back to struct.

Program 10 — File handling

Write employees to:

employees.json

and read them back.

What the developer should learn

By the end:

- Go syntax
- Packages
- Modules
- Structs
- Slices
- Maps
- Pointers
- Functions
- Errors
- JSON
- Basic file handling
- Git basics

Assignment 1 — Go Project + First REST API

Objective

Move from Go programs to a backend service.

Create:

Training Management API

Project structure

Initially keep it simple:

```
training/
├── main.go
├── go.mod
├── go.sum
│
├── handler/
├── service/
├── repository/
├── models/
│
└── config/
```

Task 1 — HTTP Server

Create:

GET /health

Response:

```
{
  "status": "ok"
}
```

Learn:

- HTTP
- Request
- Response
- Handler
- Status codes

Task 2 — First API

Create:

GET /v1/training/hello

Response:

```
{
  "message": "Welcome to Training API"
}
```

Task 3 — Logging

Introduce structured logging.

Log:

request received
request completed
error

Example:

INFO request received method=GET path=/health
INFO request completed status=200

Task 4 — Configuration

Introduce:

config
environment variables

Example:

APP_PORT=8080
APP_ENV=local

Developer should learn why secrets/configuration should not be hardcoded.

Task 5 — Graceful Shutdown

Implement:

SIGINT
SIGTERM

and gracefully shut down the HTTP server.

Expected learning

The developer should now understand:

Client
  ↓
HTTP
  ↓
Handler
  ↓
Response

Assignment 2 — Database + Repository + GET APIs

Now introduce a real database.

Use PostgreSQL

Objective

Understand how a Go backend communicates with a database.

Step 1 — Database Design

Create:

`training_department`

id
company_id
code
name
description
is_active
created_at
updated_at
deleted_at

`training_user`

id
company_id
first_name
last_name
email
password
is_active
created_at
updated_at

---

soft delete

Step 2 — Models

Create:

training/models/

Example:

```
type Department struct {
    ID          int64
    CompanyID   int64
    Code        string
    Name        string
    Description string
    IsActive    bool
}
```

Step 3 — Repository

Introduce repository pattern.

training/repository/

Interface:

```
type DepartmentRepository interface {
    GetByID(...)
    ListByCompanyID(...)
}
```

Implement database repository.

Step 4 — Pagination

Introduce:

```
type QueryFilters struct {
   Page     int
   PageSize int
   Search   string
   SortBy   string
   SortOrder string
}
```

API:

GET /v1/training/departments?page=1&page_size=20

Then:

GET /v1/training/departments?search=engineering

Step 6 — Service Layer

Create:

training/domain/

Service:

GetDepartment()
ListDepartments()

The service should:

- validate business conditions
- call repository
- map repository errors
- return domain-level errors

Step 7 — Error Handling

Create:

training/errors/

Example:

ErrDepartmentNotFound
ErrInvalidCompany

Map errors to HTTP:

Not Found → 404
Bad Request → 400
Internal Error → 500

Step 8 — APIs

Implement:

GET /v1/training/departments
GET /v1/training/departments/{id}

Real-world exercise

Ask the developer to intentionally introduce:

invalid ID
missing company ID
database unavailable
department not found
invalid pagination

Then debug each case.

This is extremely valuable for fresh developers.

Assignment 3 — Authentication

Now introduce authentication.

Objective

Understand how real backend authentication works.

Login

POST /v1/training/login

Request:

```
{
  "email": "user@example.com",
  "password": "Password@123"
}
```

Response:

```
{
  "access_token": "xxxxx",
  "token_type": "Bearer"
}
```

Tasks

1. Request validation

Use the project's:

shared/validator

Validate:

email required
email valid
password required

---

2. Password verification

Introduce password hashing.

Developer should understand:

Plain password
↓
Hash
↓
Database

Never store plain passwords.

---

3. JWT

Generate:

access token

Token should contain:

user_id
company_id
email

---

4. Configuration

JWT secret must come from:

configuration / secret

not source code.

Protected API

Create:

GET /v1/training/me

Request:

Authorization: Bearer <token>

Response:

```
{
  "id": 10,
  "email": "user@example.com"
}
```

Logout

POST /v1/training/logout

Developer must document whether logout is:

Option A:
JWT invalidation / blacklist

Option B:
No-op because JWT is stateless

The important part is understanding the trade-off.

Assignment 4 — Registration + Validation + Transaction

Now introduce more realistic backend development.

API

POST /v1/training/register

Request:

```
{
  "first_name": "Rahul",
  "last_name": "Shewale",
  "email": "rahul@example.com",
  "password": "Password@123",
  "department_code": "ENG"
}
```

Validation

Rules:

first_name → required
last_name → required
email → valid email
password → minimum length

Business Rules

Rule 1

Email must be unique.

duplicate email
↓
409 Conflict

Rule 2

If department is supplied:

department must exist
AND
department must be active

Rule 3

User and department mapping must be created atomically.

Transaction

Introduce:

tx.Begin()

Flow:

BEGIN
  ↓
Create User
  ↓
Create User-Department Mapping
  ↓
COMMIT

If something fails:

ROLLBACK

Important Exercise

Ask:

> What happens if user creation succeeds but department mapping fails?

Developer should understand why transactions exist.

Assignment 5 — Complete CRUD + Authorization

Now the developer starts working like a production backend developer.

Department APIs

Method	API	Purpose
POST	`/v1/training/departments`	Create
GET	`/v1/training/departments`	List
GET	`/v1/training/departments/{id}`	Get
PUT	`/v1/training/departments/{id}`	Update
PATCH	`/v1/training/departments/{id}/status`	Activate/Deactivate
DELETE	`/v1/training/departments/{id}`	Soft delete

Authorization

Introduce:

Authentication
↓
Who are you?
↓
Authorization
↓
What are you allowed to do?

For example:

ADMIN

MANAGER
USER

Business Rules

Implement at least these:

Rule 1

Cannot delete a department if active users are mapped.

Rule 2

Cannot deactivate the only active department of a company.

Rule 3

Normal users cannot create/delete departments.

Rule 4

Users can only access departments belonging to their company.

This introduces an important production concept:

Multi-tenant data isolation

Assignment 6 — Middleware & Request Context

Create:

middleware/
  authorization/
  request/

Implement:

RequestContextMiddleware

Context:

request_id
user_id
company_id

Request flow

Developer should understand this:

HTTP Request
  ↓
Request ID Middleware
  ↓
Authentication Middleware
  ↓
Authorization Middleware
  ↓
Handler
  ↓
Service
  ↓
Repository
  ↓
Database

Important Exercise

Authentication

> Who are you?

Authorization

> What can you do?

Request Context

> What information should be available throughout this request?

This is an important production-level concept.

Assignment 10 — Concurrency in Go

Now teach the developer what makes Go special.

Exercise 1

Process 100 jobs sequentially.

Then process them using:

goroutines

Compare execution time.

Exercise 2 — Worker Pool

Build:

10 workers
1000 jobs

Understand:

goroutine
channel
worker
producer
consumer
WaitGroup

Exercise 3 — Fan-out / Fan-in

Build:

Producer
  ↓
Workers
↓ ↓ ↓ ↓
  ↓
Aggregator

Exercise 4

Create a race condition.

Run:

go test -race ./...

Find and fix it.

This is a **must-do Go exercise**.

Assignment 11 — Performance & Database Optimization

Now teach the developer how to think beyond "the API works."

Given:

GET /departments

API is slow.

Developer must investigate:

Application
↓
Database
↓
Query
↓
Index
↓
Network
↓
Serialization

Tasks:

- Analyze SQL query
- Add appropriate index
- Compare before/after
- Implement pagination
- Avoid N+1 queries
- Understand connection pooling

Assignment 12 — Production Contribution

Theme

> **Build like an existing production module, then touch real code safely.**

Choose one small task from an existing module.

Examples:

Add validation
Add missing API
Fix error handling
Improve logging
Add unit tests
Add pagination
Fix SQL query
Add missing metric
Improve documentation

Production Contribution Rules

Before submitting:

1. Understand existing code

Developer should answer:

Where does request enter?
Where is validation?
Where is authentication?
Where is authorization?
Where is business logic?
Where is database access?
How are errors handled?
How are logs written?

---

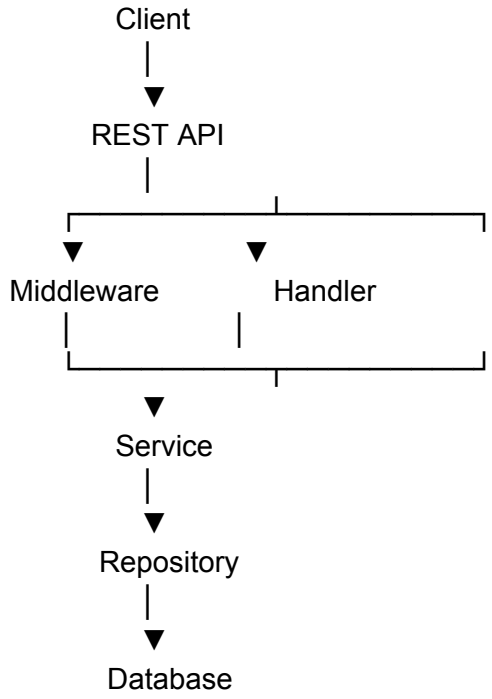
2. Don't blindly copy

Developer should explain:

Why did you implement it this way?

Final Capstone — Training Management System

At the end, developer should have:



With:

Authentication
Authorization
Validation
Transactions
Error handling
Logging
Request context
Testing
Metrics
Tracing
Concurrency
Database optimization